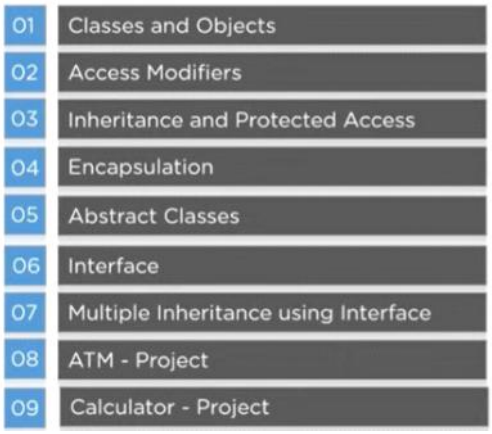
Java

Wednesday, June 3, 2026 5:01 PM

OOPS:



Classes in Objects :

1. Class is blue print of an objects.
2. Class is Logical Entity(Students, Actors)
3. Every Entity have Behavior(Functions ,actions, methods) and Attributes(Data members, properties)
4. Inside a Java program there is a class .class in java have 2 properties 1. Behavior 2. Attributes

This : Referring to the current object Properties.:

Set : to set the Value.

Get : to read the values

Object: Object is instance of class.

Access Modifiers

This modifiers are keywords are used in front of classes, variables, methods.

Access Levels

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

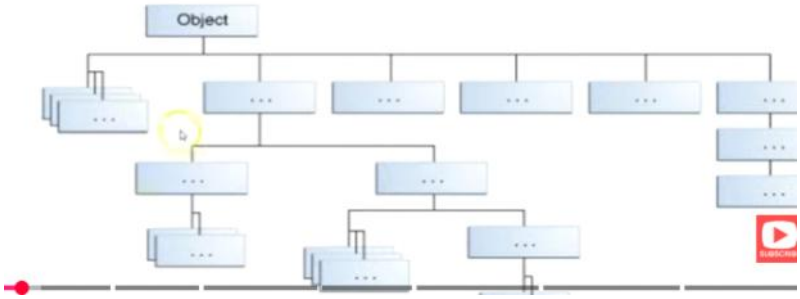
Inheritance :Extending the properties & Behavior of parent class to child class is called a Inheritance

It is also called as IS-A-Relation ship

Extend Keyword is used for Inheritance.

The Java Platform Class Hierarchy

The `Object` class, defined in the `java.lang` package, defines and implements behavior common to all classes—including the ones that you write. In the Java platform, many classes derive directly from `Object`, other classes derive from some of those classes, and so on, forming a hierarchy of classes.



Super (): To refer to the base class Constructor:

super is used to access the parent class variables, methods, and constructors from a child class.

Whenever dealing with inheritance ,whenever to initialize the super class or base class constructor ,use the super () in child class.

```
class Bicycle {

    protected int gear;
    protected int speed;

    public Bicycle(int startSpeed, int startGear) {
        gear = startGear;
        speed = startSpeed;
    }


    public void setGear(int newValue) {

class MountainBike extends Bicycle {

    public int seatHeight;

    public MountainBike(int startHeight,
                        int startSpeed,
                        int startGear) {
        super(startSpeed, startGear);
        seatHeight = startHeight;
    }

    public void setHeight(int newValue) {
```

Multiple Inheritance :

1. java does not support multiple inheritance with classes to avoid ambiguity (Diamond problem).
2. However its support multiple inheritance through interfaces..

extends -> Only one class
implements -> Multiple interfaces

Bro, **Multiple Inheritance** means:

> A child class inherits properties and methods from **more than one parent class**.

Example

```java
class A {
    void methodA() {
        System.out.println("A");
    }
}

class B {
    void methodB() {
        System.out.println("B");
    }
}

// Not allowed in Java
class C extends A, B {
}
```

Show less

Why Java doesn't support it?

Because of the **Diamond Problem**.

```java
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
}

class C extends A {
}

class D extends B, C {  // Suppose Java allowed this
}
```

Show more lines

Now if:

```java
D obj = new D();
obj.show();
```

Java won't know whether to call `show()` from path ↓ **A** or **C → A**.

This ambiguity is called the **Diamond Problem**.

How Java Achieves Multiple Inheritance?

Using **Interfaces**.

```java
interface A {
    void methodA();
}

interface B {
    void methodB();
}

class C implements A, B {

    public void methodA() {
        System.out.println("Method A");
    }

    public void methodB() {
        System.out.println("Method B");
    }
}
```

Show less

✅ Allowed because interfaces don't create the same ambiguity as multiple class inheritance.

Interview Answer

> Java does not support multiple inheritance through classes to avoid the Diamond Problem. It supports multiple inheritance through interfaces using the `implements` keyword.

Interview Answer

Q: What if two interfaces have the same method name?

✅ If they are abstract methods, implement it once in the class.

✅ If they are `default` methods, the class must override the method and resolve the conflict.

Easy memory:

```
Abstract methods -> No issue
Default methods  -> Must override
```

Single Inheritance ---> A->B

Multi-level -Inheritance --->
A child becomes a parent for another class.

A
↓
B
↓
C

3. Hierarchical Inheritance
Multiple child classes inherit from the same parent.

```
  A
 / \
 B C
```

Use case :

Exactly bro ✅

If all your citizen-related entities have some common fields, you can keep them in a parent class and extend them.

Example:

```java
class BaseCitizen {

    Long id;
    String createdBy;
    Date createdDate;
    Date updatedDate;
}
```

Then:

```java
class Applicant extends BaseCitizen {

    String applicantName;
    String aadharNumber;
}
```

```java
class Beneficiary extends BaseCitizen {

    String beneficiaryName;
}
```

Now Applicant and Beneficiary automatically get:

Encapsulation: Wrapping the data members and member functions into a single unit and restricting the direct access using private variables.
To protect the properties of class.
It is one of the security feature.

Data Hiding and Security

1. How to Implement the Encapsulation:
   1.make it private for data members or properties.
   2.create setters and getters

For example there is a student details make it as private. but it only accessible for invidulator so by using setters and getters

Capsul: Pack inside like a tube.

1. Encapsulation → Capsule / ATM Machine
👉 Analogy: Think of an ATM machine

You insert your card, enter a PIN, and withdraw money.
You don't know how the ATM processes transactions internally.
The internal data (your bank balance) is hidden and protected.

☑ Key idea:
You can only interact through controlled methods (PIN, buttons), not directly with the system.

Bro, think of **Encapsulation = Data Hiding + Controlled Access.**

Without Encapsulation ❌

```java
class Citizen {
    public int age;
}

Citizen c = new Citizen();
c.age = -10;   // Wrong data
```

Anyone can modify the data.

With Encapsulation ✅

```java
class Citizen {

    private int age;

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }

    public int getAge() {
        return age;
    }
}
```

```java
Citizen c = new Citizen();

c.setAge(25);
System.out.println(c.getAge());
```

Output:

```
25
```

Now invalid data is prevented.

Your Eligibility Project Example

```java
class Citizen {

    private String eligibilityStatus;

    public void setEligibilityStatus(String status) {
        if(status.equals("APPROVED") ||
           status.equals("REJECTED")) {
            eligibilityStatus = status;
        }
    }

    public String getEligibilityStatus() {
        return eligibilityStatus;
    }
}
```

Show less

Nobody can directly change the status.

They must use:

```java
citizen.setEligibilityStatus("APPROVED");
```

Abstraction: Hiding Something

1. Hiding the Implementation data providing the essential data.
2. Abstract classes can't be instantiated but they can be subclassed.
3. Abstract method is method that is declared without an implementation(without braces, and followed by semicolon)

EX: ATM Machine

```
abstract void moveTo(double deltaX, double deltaY);
```

4. Abstract is a key word which are used in front of Class, method

5. While using the abstract keyword Infront of class that whole class become an abstract class.
6. While you mentioned abstract class at least one abstract method is mandatory in that class.

```java
public abstract class GraphicObject {
    int x, y;

    void moveTo(int newX, int newY) {
        System.out.println("move to x :" +x+" and y:"+y);
    }

    abstract void draw();
    abstract void resize();
}
```

```java
public class Circle extends GraphicObject {
    void draw() {
        System.out.println("Drawing a circle");
    }
    void resize() {
        System.out.println("Resizing a circle");
    }
}
```

7. Abstract class also have concrete methods.
8. An abstract class extend the another abstract class.
9. Abstract class can't be initialized explicitly .they can only be initialized through internally mechanism with child class

```java
public static void main(String[] args) {
    GraphicObject circle = new Circle();
    circle.draw();
    circle.resize();

    GraphicObject rectangle = new Rectangle();
    rectangle.draw();
    rectangle.resize();
}
```

child classes

Bro, for your **Integrity Eligibility System**, a simple and realistic **Abstract Class** use case is:

Example: Different Schemes

All schemes have some common functionality.

```java
abstract class Scheme {

    String citizenId;

    abstract boolean checkEligibility();

    void displayMessage() {
        System.out.println("Eligibility Check Started");
    }
}
```

Child classes:

```java
class PensionScheme extends Scheme {

    boolean checkEligibility() {
        return true;
    }
}

class HealthScheme extends Scheme {

    boolean checkEligibility() {
        return true;
    }
}
```

Why Abstract Class?

- Every scheme needs `checkEligibility()`.
- But eligibility logic differs for each scheme.
- Common code stays in the abstract class.
- Specific logic is implemented by child classes.

Abstract class is used when multiple classes share some common behavior, but each class needs its own implementation of certain methods

Pillar	Meaning
Encapsulation	Protect data
Inheritance	Reuse code

Polymorphism	Multiple behaviors
Abstraction	Hide complexity

☑ Quick Memory Trick

Pillar	Real-life Example	Core Idea
Encapsulation	ATM machine	Hide & protect data
Inheritance	Parent → Child	Reuse features
Polymorphism	Same person behavior	Different outcomes
Abstraction	Driving a car	Hide complexity

Interface : An interface is a blue print that contains abstract methods.
Interface is a contract that tells a class what methods it must implement.

1. An interface contains only constants, Method signatures, default methods static methods and nested types.
2. Interfaces cannot be instantiated they can only be implemented by classes or extended by other interfaces.

```
interface Bicycle {

    void applyBrake(int decrement);

    void speedUp(int increment);
}
```

In interface variables are must and should be public static final.

Interface Variable = Constant

public -> Accessible everywhere
static -> Belongs to interface
final -> Cannot change

1. Why do we use Interfaces

-> Achieve abstraction
-> Support multiple Inheritance
-.Loose Coupling
->improve maintainability and testability

1. Can an interface have method implementation ?

-> Yes from java 8 onwards.

1. Difference between abstract class and Interface?

Interface	Abstraction
1. Supports multiple inheritance	Does not support multiple inheritance
2. No constructor	Can have Constructor
3. Variables are public static final	Can have instance variables.
4. Interface Keyword is used	Abstract keyword

1. Can we create object for an interface?

No,

But Animal a =new Dog();

Can an interface extend the another interface?
-> Yes

Can a class implement multiple interface ?
-> Yes :

```
Class Test implements A, B
{
}
```

This is how java achieves multiple Inheritance.

What are default methods ?

-> methods with implementation inside the interface

What are functional Interface?

An interface have only one abstract method.

EX: Runnable , Callable , Comparator

What is markable interface ?

An interface with no methods

Can Interface variables can be changed ?

No, All variables are public static final

What happens if two interface have same default method ?

The implementation class must override that method to resolve conflict.

```java
interface A
{
    default void show()
    {
        System.out.println("A");
    }
}
interface B
{
    default void show ()
    {
        System.out.println("B");
    }
}
class Test implements A,B
{
    public  void show ()
    {
        A.super.show();
        B.super.show();
        System.out.println("heeelo");
    }
}
class Main {
    public static void main(String[] args) {
        System.out.println("Start small. Ship something.");
        Test t =new Test();
        t.show();
    }
}
```

```
Start small. Ship something.
A
B
heeelo

=== Code Execution Successful ===
```

Can an interface have constructor ?

->No

Why do spring projects prefer interface for services?

1. Loosely coupling
2. Supports Dependency Injection.

Yes bro, that's a **good real-time explanation**. Just make it a little more interview-friendly:

> "In the Integrity Eligibility System, there are multiple schemes and each scheme has its own eligibility rules. We can define a common interface like `EligibilityChecker` with a method `checkEligibility()`. Then each scheme implements that interface and provides its own eligibility logic. This gives flexibility and makes the application easy to maintain and extend."

Simple Flow

```
EligibilityChecker (Interface)
        |
   --------------------
   |                  |
PensionScheme    HealthScheme
   |                  |
Own Logic        Own Logic
```

Why interface why not abstract?

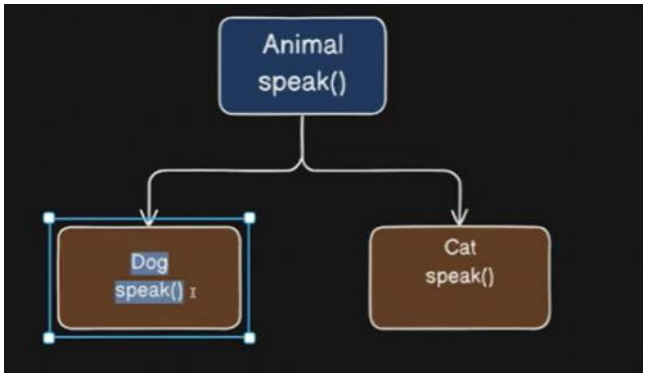
1. Achieve loosely coupling and supports multiple inheritance
   Abstract classes are used only when multiple classes share common state or implementation."

==========

1. Polymorphism
   ==========

. Any object in a class to perform particular task in multiple methods is called polymorphism.

One method, many behaviors.



Animal class extending the both dog and cat Classes.

1 method--Speak ()---> gives to 2 different out puts--> 1. Dog sound 2. Cat Sound.

1. **Dynamic binding or late binding or Run time polymorphism.[the method that gets executed is determined at runtime ,not a compile time.**

Animal class object create at run time it will resolve to call method at run time .
It is also called as later binding because after execution of program then only It will executed at run time

```
Animal cat = new Cat();
cat.speak();
```

When the method to execute is decided at runtime based on the actual object

To achieve runtime polymorphism through the Method overriding. Means speak () extending the dog and cat and change the body (i.e. is overriding).

Method overriding: when child class provides its own implementation of a method that is already defined in the parent class.

1. Method name should be the same
2. Parameters must be the same.
3. Return type must be the same or covariant.
4. Access modifier cannot be more restrictive.
5. Final, static and private methods cannot be overridden.
6. Happens only in inheritance.

2. Method Overloading(Compile time polymorphism/static binding/Early binding .):

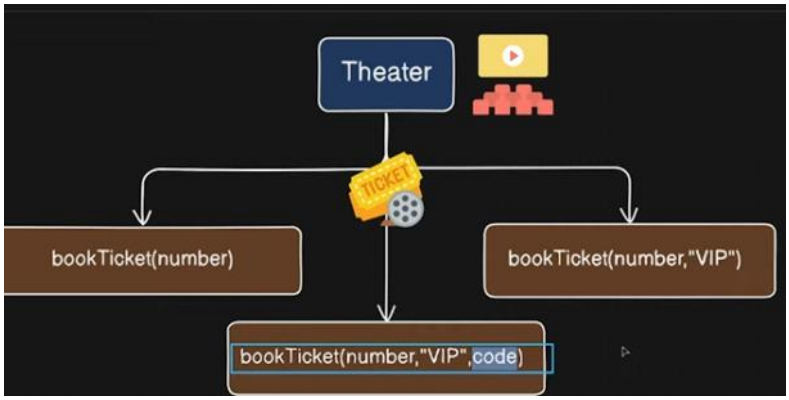
   -> Same method name with different arguments.
   Method overloading is compile time polymorphism.

   Based on 1. number of arguments
   2. Type of arguments
   3. Order of parameters.

Different Parameter List:

The **number, type, or order of parameters** must differ.

- **Number:** `add(int a)` vs. `add(int a, int b)`
- **Type:** `add(int a)` vs. `add(double a)`
- **Order:** `add(int a, double b)` vs. `add(double a, int b)`




```
class Theatre {  2 usages
    void bookTickets(Integer count){  1 usage
        System.out.println("Booked " + count + " tickets.");
    }
    void bookTickets(Integer count,String seatType){  1 usage
        System.out.println("Booked " + count + " " + seatType + " tickets.");
    }
    void bookTickets(Integer count,String seatType, String discountCode){  1 usage
        System.out.println("Booked " + count + " " + seatType + " tickets with discount code: " + di
}
```

Overloading = Compile-Time Polymorphism (decision made at compile time).

Overriding = Runtime Polymorphism (decision made at runtime).

Advantages:

1. Code Reusability
2. Easier maintenance

Constructors

Constructors is a special method is used to initialize an object.

Same name as class name

Does not have return type.

Types of constructors:

1. Default constructor:

If you don’t create any constructor java provides one automatically.

2. No-Argument constructor:
it don’t have arguments.

3. Parameterized Constructor:

4. Copy Constructor:

Copy Constructor is a constructor that creates a new object by copying the values from an existing object of the same class.

Creating duplicate data objects without rewriting values.

```java
class Student {
    String name;
    int age;

    // Normal constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }

    // Copy constructor
    Student(Student s) {
        name = s.name;
        age = s.age;
    }
}

public class Main {
    public static void main(String[] args) {
        // Original object
        Student s1 = new Student("Om", 22);

        // Copy object
        Student s2 = new Student(s1);

        System.out.println(s1.name + " " + s1.age);
        System.out.println(s2.name + " " + s2.age);
    }
}
```

Constructor Chaining :

Constructor chaining means calling one constructor from another constructor.

1. With in the same class using this()
2. Between the parent class and child class using super()

1. Can have a constructor be private.

Used in single ton design pattern

2. Can a constructor be final ?

Constructors cannot be inherited or overridden so final is not allowed.

Difference between super() and this():

This()	Super()
1. Calls same class constructor	Calls parent class constructor
2. used within the class	Used in Inheritance
3. Must be first statement	Anywhere

What happen if both this() and super() are written in same constructor.

Compilation Error.

Only one of them can appear , and it must be first statement.

1. Can constructor Inherited?

Constructors are not inherited ,but child classes can invoke parent class constructors using super().

this.name // Current class variable
super.name // Parent class variable

this.show() // Current class method
super.show() // Parent class method

this() // Current class constructor
super() // Parent class constructor

What is object class

-> 1. object class is the super class of all classes in java.

Available in java.lang

Important methods of Object class.

1. To String(),---Returns a String representation of an object.
2. Equals()---used to compare to objects.
3. Hash code()->Returns hash value of an object.
4. Get class()-> returns run time class information
5. Clone()-> creates a copy of an object.
6. Wait(), notify(),Notify All()-> used for thread communication.

Why object class-> provides common methods such as tostring, equals()….. Which can be used by all java objects.

Object cloning

It is the process of creating an exact copy of existing object.

Class Employee implements Cloneable **[1. implement**]

{
Public object clone() throws exception **[2. override clone()**]
{
return super. Clone() **3.call super ()**

}
}

Main ()

Employee e2 =(Employee) e1. clone();

Example

```java
class Citizen implements Cloneable {

    String name = "Omprakash";

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class Test {

    public static void main(String[] args)
            throws CloneNotSupportedException {

        Citizen c1 = new Citizen();

        Citizen c2 = (Citizen) c1.clone();

        System.out.println(c1.name);
        System.out.println(c2.name);
    }
}
```

Show less

Shallow copy : Shared address--> object reference are shared.

Shallow Cloning

> Copies the object, but referenced objects are shared.

Example

```java
class Address {
    String city = "Hyderabad";
}

class Citizen implements Cloneable {

    String name = "Omprakash";
    Address address = new Address();

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

```java
Citizen c1 = new Citizen();
Citizen c2 = (Citizen) c1.clone();

c2.address.city = "Bangalore";

System.out.println(c1.address.city);
```

Output

```
Bangalore
```

Why?

Because both objects point to the **same Address object**.

```
Citizen c1 -----> Address
Citizen c2 -----> Address
```

Object copied

Referenced objects not copied

Deep Copy : Separate Address--> objects are copied separately.[so changes in one object do not affect the other]

Deep Cloning

> Copies both the object and all referenced objects.

Example

```java
class Address implements Cloneable {

    String city = "Hyderabad";

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

class Citizen implements Cloneable {

    String name = "Omprakash";
    Address address = new Address();

    public Object clone() throws CloneNotSupportedException {

        Citizen c = (Citizen) super.clone();

        c.address = (Address) address.clone();

        return c;
    }
}
```

```java
Citizen c1 = new Citizen();
Citizen c2 = (Citizen) c1.clone();

c2.address.city = "Bangalore";

System.out.println(c1.address.city);
```

Output

```
Hyderabad
```

✅ Separate Address objects are created.

```
Citizen c1 -----> Address1

Citizen c2 -----> Address2
```

Easy Diagram

Shallow Clone

```
Original -----> Address
Clone    -----> Address
```

Same Address object.

Deep Clone

```
Original -----> Address1
Clone    -----> Address2
```

Different Address objects.

```
Shallow Clone
 -> Copy Object
 -> Share References

Deep Clone
 -> Copy Object
 -> Copy References
```

Up Casting.

Converting a **child class** object into a **parent class** reference

Why upcasting --To write generic, flexible code

1. To Achieve Run time Polymorphism
2. Flexible Code :

2. Flexible Code

Without upcasting:

```java
Dog d = new Dog();

```

Can only store Dog objects.

With upcasting:

```java
Animal a = new Dog();
Animal a2 = new Cat();
Animal a3 = new Lion();
```

One parent reference can handle many child objects

```java
class A
{

    public   void m1()
    {
        System.out.println("M1() is called");
    }
}
class B extends A
{
    public  void m2()
    {
        System.out.println("m2() is called");
    }
}
class Main {
    public static void main(String[] args) {
        System.out.println("Start small. Ship something.");
        A a=new B();// UPCASTING GIVING CHILD CLASS OBJECT INTO PARENT CLASS REFERENCE.
        a.m1();

    }
}
```

Down casting

Converting a **parent reference** back to a **child reference**

```java
class A
{

    public   void m1()
    {
        System.out.println("M1() is called");
    }
}
class B extends A
{
    public  void m2()
    {
        System.out.println("m2() is called");
    }
}
class Main {
    public static void main(String[] args) {
        System.out.println("Start small. Ship something.");
        A a=new B();// UPCASTING GIVING CHILD CLASS OBJECT INTO PARENT CLASS REFERENCE.
        a.m1();
       /// a.m2(); //we cant acess the child class methods so we do Down casting
        //Down casting :

        B b=(B)a;//Down casting

        b.m2();

    }
}
```

```
Start small. Ship something.
M1() is called
m2() is called

=== Code Execution Successful ===
```

Feature	Upcasting	Down casting
Direction	Child → Parent	Parent → Child
Casting	Automatic ☑	Manual ☑
Safety	Safe ☑	Risky ⚠
Access	Parent methods only	Child methods

Why Down casting?

Reason: To access specific features of a child class

Association

Association is a relationship between two independent classes where one object uses or is connected to another object.

Types of Association (basic understanding)

One-to-One → One person has one passport
One-to-Many → One teacher teaches many students

```java
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

class Teacher {
    String name;

    Teacher(String name) {
        this.name = name;
    }

    // Association: Teacher uses Student
    void teach(Student s) {
        System.out.println(name + " teaches " + s.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Om");
        Teacher t1 = new Teacher("Suresh");

        t1.teach(s1);   // association
    }
}
```

Example: Citizen submits Eligibility Request

```java
class Citizen {

    String name;

    Citizen(String name) {
        this.name = name;
    }
}

class EligibilityRequest {

    Citizen citizen;

    EligibilityRequest(Citizen citizen) {
        this.citizen = citizen;
    }

    void display() {
        System.out.println("Citizen Name: " + citizen.name);
    }
}

public class Main {

    public static void main(String[] args) {

        Citizen citizen = new Citizen("Omprakash");

        EligibilityRequest request =
                new EligibilityRequest(citizen);
```

```
        EligibilityRequest request =
                new EligibilityRequest(citizen);

        request.display();
    }
}
```

Show less

Output

```
Citizen Name: Omprakash
```

Aggregation

A relationship where one object uses another object, but both objects can exist independently.

Real-Life Example

Department and Teacher

A department has teachers
But if the department is removed → teachers still exist

☑ So, both are independent

```java
class Teacher {
    String name;

    Teacher(String name) {
        this.name = name;
    }
}

class Department {
    String deptName;
    Teacher teacher;  // aggregation

    Department(String deptName, Teacher teacher) {
        this.deptName = deptName;
        this.teacher = teacher;
    }

    void show() {
        System.out.println(deptName + " department has " + teacher.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Teacher t = new Teacher("Suresh");
        Department d = new Department("CSE", t);

        d.show();
    }
}
```

Citizen ↔ EligibilityRequest = Association

Department HAS-A Employee = Aggregation

House HAS-A Room = Composition

Composition

It is a Strong Has-A Relation
Where one object is completely dependent on the other object. If the parent object is destroyed ,the child object is also destroyed.

House and Rooms

A house has rooms 🏠
If the house is destroyed → rooms also don't exist

☑ So, dependent relationship

```java
class Room
{
    public void display ()
    {
        System.out.println("Room Created");
    }
}

class House
{
    public void showRoom()
    {
        Room room=new Room();
        room.display();
    }
}
class Main {
    public static void main(String[] args) {
House h=new House();

h.showRoom();

    }
}
```

```
Room Created

=== Code Execution Successful ===
```

Feature	Aggregation 🔗	Composition 🧱
Relationship	Weak	Strong
Dependency	Independent	Dependent
Example	Teacher–Dept	Room–House
Ownership	No	Yes

Final One-Line Answers

Aggregation → Objects are connected but independent
Composition → Objects are connected and dependent

Association → Uses
Aggregation → Has (independent)
Composition → Has (dependent)

Association is a general relationship,
Aggregation is a weak has-a relationship (independent), and
Composition is a strong part-of relationship (dependent).

Interview Questions.

3. Can we override a private method

No , We cannot override a private method in java.
Child classes cannot see or access it.
Since overriding requires access, it cannot be overridden.
Because a private method belongs only to that class and is **not visible** to child classes.
If the child class cannot see the method, it cannot override it.

```java
class Parent {
    private void show() {
        System.out.println("Parent show()");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child show()");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
}
```

Show less

Output

```
Child show()
```

Can we override static method

No, Static methods are belongs to classes ,not to the object therefore, method overriding(which is run time polymorphism) does not apply the static methods.

Instance methods--> Overridden
Static methods --> Method Hidden.

```java
class A
{

    public static void m1()
    {
        System.out.println("M1() is called");
    }
}
class B extends A
{
    public static void m1()
    {
        System.out.println("m2() is called");
    }
}
class Main {
    public static void main(String[] args) {
        System.out.println("Start small. Ship something.");
        A a=new B();
        b.m1();
    }
}
```

```
Start small. Ship something.
M1() is called

=== Code Execution Successful ===
```

4.

Method Hiding

When a child class defines the same static method:

```java
class Child extends Parent {

    static void show() {
        System.out.println("Child");
    }
}
```

This is called:

> **Method Hiding**, not Method Overriding.

5. So while oveeride the static methods child class static methods are moth Hiding not method overeriding.

Overriding → Runtime
Static Methods → Compile Time

6. Can constructors be overridden ?

No. Constructors can't be overridden .because they are not inherited by child class. And are used only for object initialization , not for polymorphism .

Method Overriding ☑
Static Method Override ❌ (Method Hiding)
Private Method Override ❌
Constructor Override ❌
Constructor Overload ☑

6. Difference: Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Both	Mostly abstract
Variables	Allowed	static final
Inheritance	single	multiple

In project Oops Usage.

Encapsulation:

“We used encapsulation in entity classes like User, Product, where fields are private and accessed using getters/setters.”

Polymorphism:

“We used polymorphism in service layer, where different payment methods implement the same interface.”

Example:

```java
interface PaymentService {
    void pay();
}

class CreditCardPayment implements PaymentService {
    public void pay() {
        System.out.println("Paid via card");
    }
}

class UpiPayment implements PaymentService {
    public void pay() {
        System.out.println("Paid via UPI");
    }
}
```

Show less

Abstraction:

We used abstraction by defining interfaces such as UserService and implementing them in service classes.”

Example:

```java
interface UserService {
    void createUser();
}

class UserServiceImpl implements UserService {
    public void createUser() {
        // logic
    }
}
```

“In my project, we used encapsulation in entity classes, inheritance for common base models, polymorphism in service implementations like payment handling, and abstraction using interfaces in service layers. This helped us build a modular and scalable backend system.”

Explain SOLID with real example

Design eligibility logic for 10+ schemes without breaking existing code.

abstraction So 1. Brief Architecture project
2. Write down the modules
3. Daily 1 hr. 1 module preparation
4. Then established the microservices

28. What is instance of?

instanceof Operator

Java provides the instanceof operator to check whether an object belongs to a particular class.

```java
class Animal {
}

class Dog extends Animal {
}

public class Main {

    public static void main(String[] args) {

        Dog d = new Dog();

        System.out.println(d instanceof Dog);
        System.out.println(d instanceof Animal);
    }
}
```

Show less

Output

```
true
true
```

1. lEquals and ==
2. Final Keyword

1.
2. Exception handling
3. JVM JDK JRE Garbage Collection.

Design patterns.

JVM Architecture:

```
Java Program (.java)
        ↓
Compiler (javac)
        ↓
Bytecode (.class)
        ↓
       JVM
   ┌──────────────────┐
```

```
| Class Loader  |
| Runtime Data  |
| Execution Eng |
| Native Interface
└──────────────────┘
     ↓
  Output
```

3. Class Loader

Loads bytecode into JVM memory.

Types:

```
1  Bootstrap Class Loader
2  Extension Class Loader
3  Application Class Loader
```

Interview Answer:

> Class Loader loads class files into JVM memory.

🔹 15. How does JVM memory work during execution?

✅ **Answer:**

- Class loads → Method Area
- Object creation → Heap
- Method calls → Stack
- Execution tracking → PC Register

JVM Executes Bytecode. It consists:

1. Class Loader[Class Loader loads the classes]
2. Runtime Memory area(heap,stack,Method area)[memory areas store data]
3. Execution Engine[Execution Engine runs Bytecode]
4. Garbage collector[garbage collector removes unused objects]

==========
Class Loader
==========

1. Class Loader is responsible for loading .class files into JVM memory.

Step-by-step:

Application ClassLoader receives request
It asks its parent → Extension Class Loader
Extension asks its parent → Bootstrap Class Loader
Bootstrap tries to load class

Works using parent delegation model ->Child asks parent first before loading class
Application → Extension → Bootstrap

2. Why classloader hierarchy is neede ?

- Security
- Avoid duplicate Loading
- Maintain Consistency.

3. Bootstrap Class Loader -> Loads Core java classes ->

Ex: java.lang.String
java.lang.Object
java.lang.System
java.util.List
java.util.ArrayList

4. Application ClassLoader -> Loads your custom classes
Ex: Main.class, Employee.class

==========

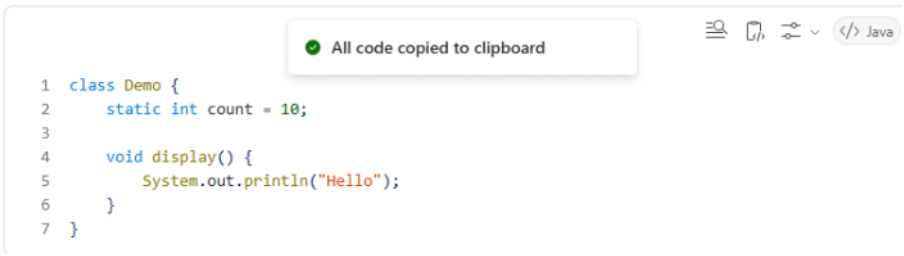
Runtime Memory area
==========

1. Runtime Memory Area = memory created when JVM runs a program
   It stores:

- Class Definition
- Static Variables
- Method code (Byte code)

2. Method Area is a part of JVM memory that stores class metadata, static variables, and method bytecode shared across all objects.

Simple Example



Class structure of-> Demo
Static variable → count
Bytecode of method → display()

Class Structure:
Static Variable:
Variables declared with static
Shared across all objects of the class
Method Code(Byte code)

Method bodies
Constructors
Static blocks

Shared Memory :

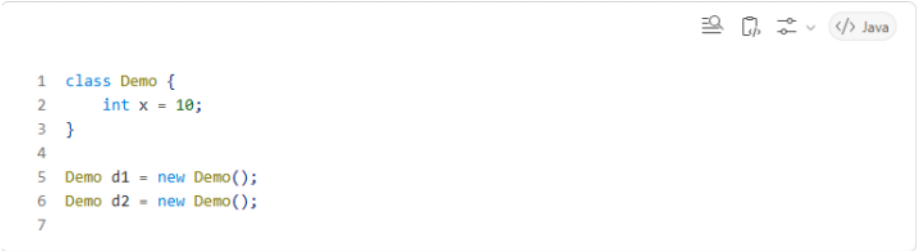
```
class Test {
    static int x = 100;
}

Test t1 = new Test();
Test t2 = new Test();
```

All objects share the same x variable

Heap Memory area

1. Heap memory is used to store objects created using new keyword



d1 and d2 objects → stored in Heap memory
Each object gets its own copy of variables (x)

What Heap Stores

- Objects
- Instance variables (non-static variables)
- Arrays

| Heap Memory | Method Area |

| ------------------- | ------------------------ |
Stores objects	Stores class data
Instance variables	Static variables
One copy per object	One copy per class
Created using `new`	Created at class loading

Stack Memory :

- Stack memory is a part of JVM used to store method calls and local variables, and it works in a LIFO manner for each thread.

```java
class Demo {
    void show() {
        int x = 10;   // stored in stack
        System.out.println(x);
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        d.show();
    }
}
```

👉 What happens in Stack

1. `main()` starts → **stack frame created**
2. `d.show()` is called → new **stack frame created**
3. Variable x stored in that frame
4. When method ends → frame is **removed automatically**

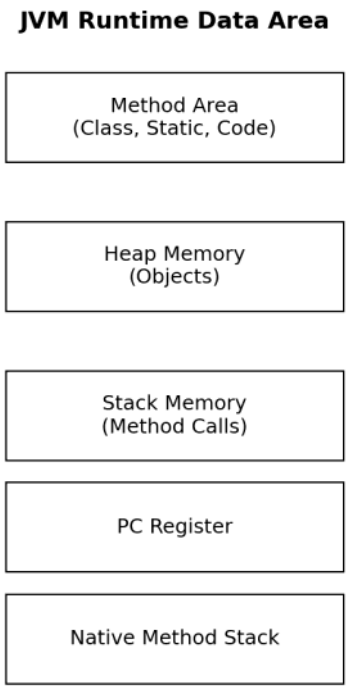
✅ What Stack Stores

- Method calls
- Local variables
- Function parameters
- Partial results during execution

Stack Memory	Heap Memory
Stores method calls	Stores objects
Local variables	Instance variables
Thread-specific	Shared between threads
Fast	Slower than stack
Auto deleted	Garbage collected

☑ Easy Trick to Remember:

Stack → Methods & Execution
Heap → Objects
Method Area → Class & Static



✅ Quick Understanding of Diagram

🔹 1. Method Area

- Stores **class data, static variables, method code**
- ✅ Shared

🔹 2. Heap Memory

- Stores **objects and instance variables**
- ✅ Shared + Garbage Collected

🔹 3. Stack Memory

- Stores **method calls, local variables**
- ✅ Thread-specific

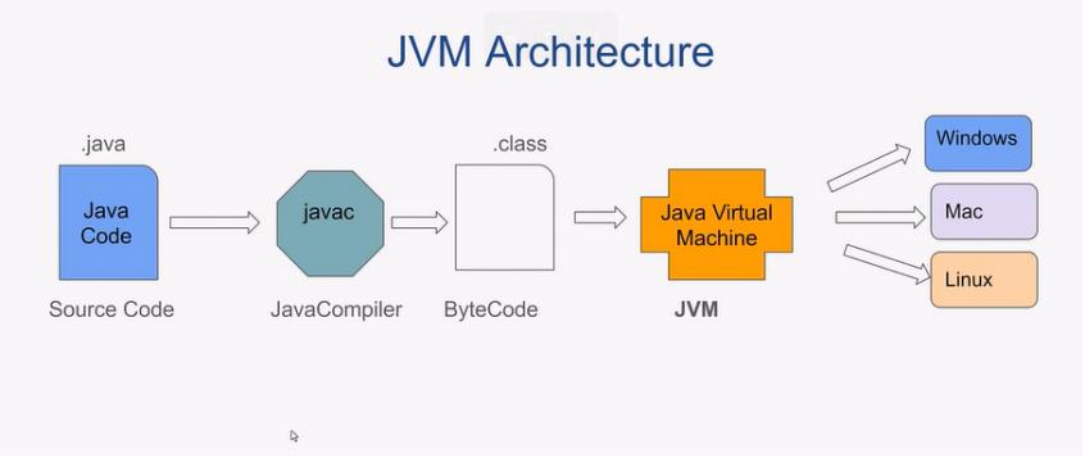
🔹 4. PC Register

- Stores **current instruction address**
- ✅ Thread-specific

🔹 5. Native Method Stack

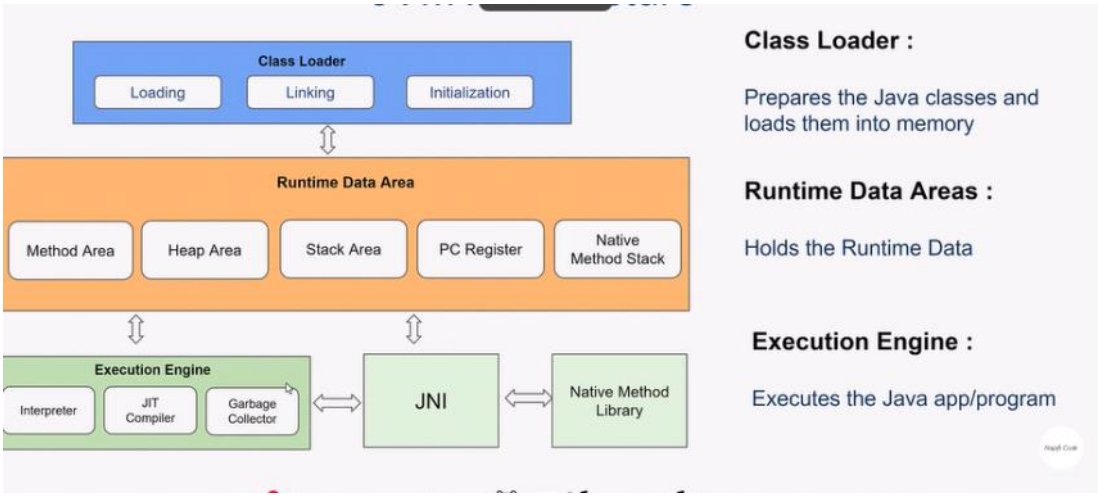
- Used for **C/C++ native methods**
- ✅ Thread-specific

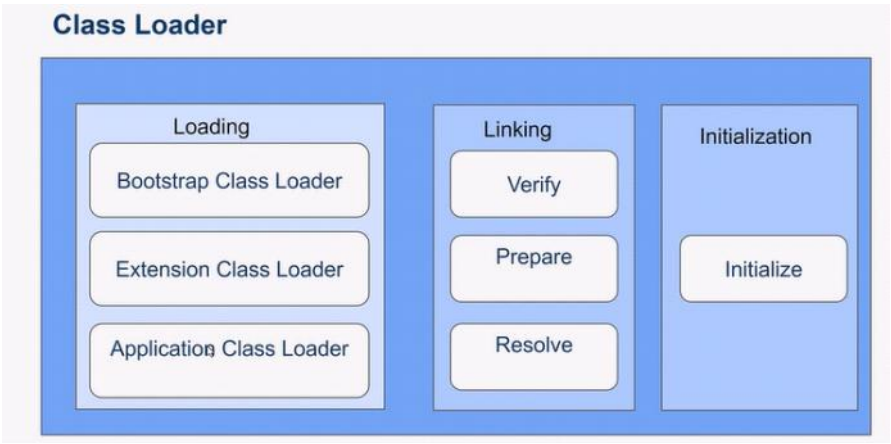
We write a java code (.java) and give to compiler javac then compiler convert into byte code(.class)and then byte code gives to JVM .then JVM converts machine understanding Language. with this code run in any machine(windows, Linux).



so, write once runs anywhere.

So, Java is a platform Independent(Use bytecode and execute anywhere)





1. To load the entire byte code in the JVM ecosystem.
2. Bootstrap : It is the parent of all class Loader ,It is the loading classes in the JVM.
3. It load the predefined classes(String class..)
4. Extension class Loader : It is the child of the bootstrap loader.
5. It is responsible for loading java extension libraries.
6. Application class loader: It is a child of the extension of class loader.
7. Whatever classes you have coded(Demo.java) apart from the predefined class

Linking:

1. After the class loaded, the linking process is executed.

2. Verification : verify the byte code(Eg: Invalid bytecode)
3. Preparation: Prepared for allocation memory
4. Resolution :Resolution is the process of converting symbolic references into actual memory addresses during linking
   Demo → symbolic reference for class
   show() → symbolic reference for method
    0x7a12f3 = actual memory address of object

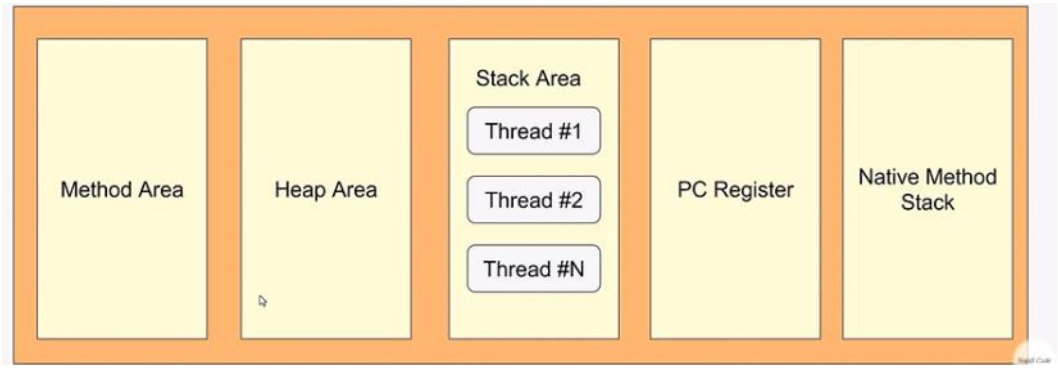
Initialization

1. Once a class has been loaded and linked , the initialization phase is triggered. During initialization:
1. Static variables are assigned their variables.
2. Static blocks are executed

JVM Runtime Data Area:

Whatever code which is getting loaded entire code kept in Method area(Class-level data like method code, constans and meta data)
Whenever object is created then the memory for the object will be allocated on the Heap area.
If there is a static variable during the class loading itself memory gets allocated on the Heap memory

If there are any methods or anything which is getting executed in your java code it happening on the stack area itself.
Stack : Local variables, method calls ,blocks, constructors
Pc Register(Program Counter): It Maintains address of every Instruction.
Native Method Stack :Stores Native code(like c Or C+)



Execution Engine:

Basically JVM use interpreter to execute byte code into machine understandable language.
Interpreter converting Byte code into binary Line by line
JIT is used to support the Interpreter(whenever frequent code calls)[it coverts entire code to binary at once]
Garbage collector : whenever In Heap memory there is an object and there is no reference variable automatically in java garbage collector will be used to deallocate the memory for the object.

Handling Native code : sometimes you need to execute code that is not written in java(C, C+) .java allows this through the java native interface .which allows java to interact with native code .

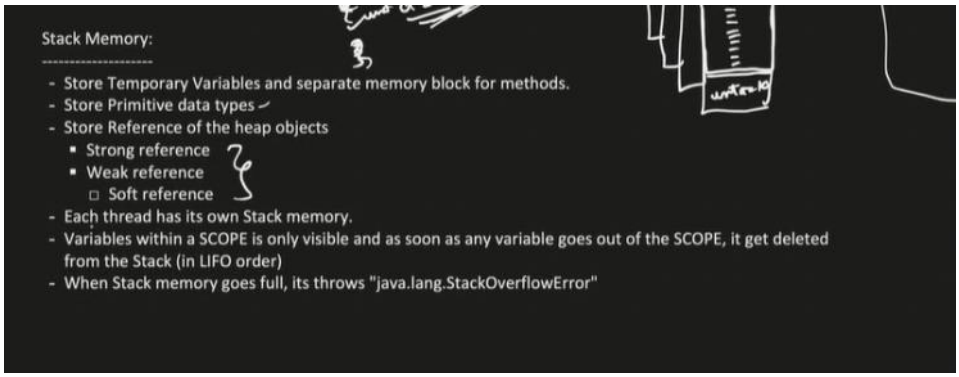
- **Class Loader**: Loads the bytecode into memory.
- **Runtime Data Areas**: Stores data like objects (heap), local variables (stack), class metadata (method area).
- **Execution Engine**: Executes the bytecode using the interpreter and JIT compiler.
- **Garbage Collector**: Automatically reclaims memory used by unreferenced objects.
- **Native Methods**: Enables interaction with non-Java code via JNI.

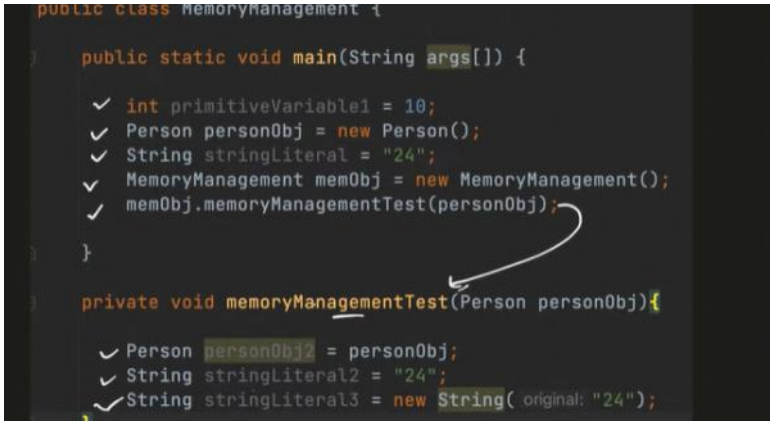
Shreyansh:

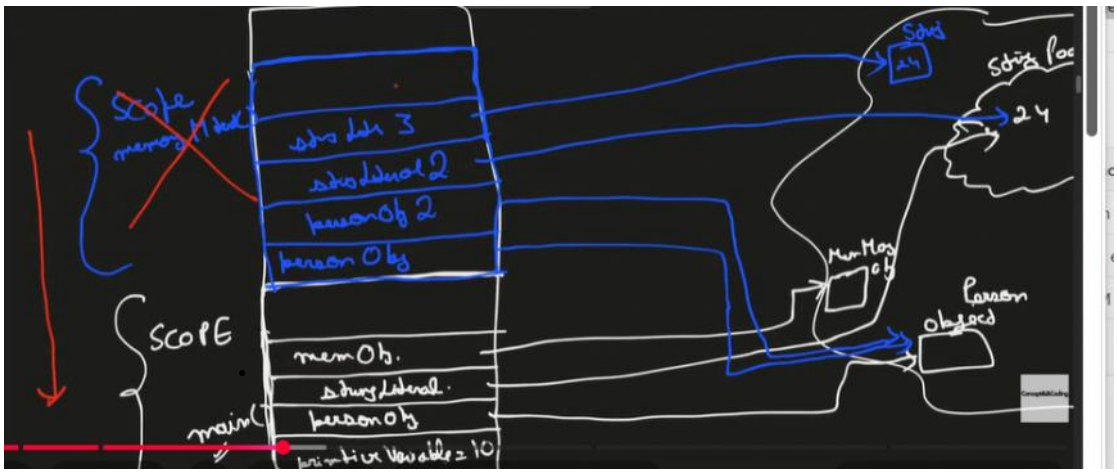


Shreyansh

1. In stack we will stored Local variables ,reference of the objects.
2. But all threads share us common heap memory.









Heap Memory:

- Store Objects.
- There is no order of allocating the memory.
- Garbage Collector is used to delete the unreferenced objects from the heap.
  - Mark and Sweep Algorithm
  - Types of GC:
    - Single GC
    - Parallel GC
    - CMS (concurrent Mark Sweep)
    - G1
- Heap memory is shared with all the threads.
- Heap also contains the String Pool.
- When Heap memory goes full, its throws "java.lang.OutofMemoryError"
- Heap memory is further divided into:
  - Young Generation (minor GC happens here)
    - Eden
    - Survivor
  - Old/Tenured Generation (major GC happens here)
  - Permanent Generation

Reference Type	GC Behavior	Use Case
Strong	Never collected (if referenced)	Default usage
Soft	Collected when memory low	Caching
Weak	Collected anytime	WeakHashMap
Phantom	After object is dead	Cleanup tracking

```
.java File
 ↓
Compiler (javac)
 ↓
.class File (Bytecode)
 ↓
Class Loader
 ↓
Runtime Data Areas
 ↓
Execution Engine
 ↓
Native Method Interface
 ↓
Native Libraries
```

1. Java Source File

```java
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Saved as:

```
Test.java
```

2. Compiler (javac)

Converts:

```
.java
```

into:

```
.class
```

(Bytecode)

```
Test.java
     ↓
javac
     ↓
Test.class
```

3. Class Loader

Loads bytecode into JVM memory.

Types:

```
Bootstrap Class Loader
Extension Class Loader
Application Class Loader
```

Interview Answer:

> Class Loader loads class files into JVM memory.

3. To load the entire byte code in the JVM ecosystem.
4. Bootstrap : It is the parent of all class Loader ,It is the loading classes in the JVM.
5. It load the predefined classes(String class..)
6. Extension class Loader : It is the child of the bootstrap loader.
7. It is responsible for loading java extension libraries.
8. Application class loader: It is a child of the extension of class loader.

9. Whatever classes you have coded(Demo.java) apart from the predefined class

4. Runtime Data Areas

Most important part.

Method Area

Stores:

```
Class Information
Static Variables
Methods
```

Example:

```java
static int count = 10;
```

Stored in Method Area.

Heap Area

Stores Objects.

```java
Citizen c =
    new Citizen();
```

Object goes into Heap.

Most interviewers ask:

Where are objects stored?

✅ Heap Memory

Stack Area

Stores:

```
Method Calls
Local Variables
`
```

Example:

```java
public void display() {

    int age = 25;
}
```

age stored in Stack.

The **PC Register** stores the address of the current/next instruction.

Example:

Java

```
System.out.println("Hello");
System.out.println("Bro");
```

Execution:

```
PC → Line 1 execute
PC → Line 2 execute
```

Interview Definition

> PC Register is a small memory area that stores the address of the currently executing JVM instruction.
> Each thread has its own PC Register.

Easy Memory Trick

Component	Remember As
PC Register	"Which instruction should execute next?"
Java Stack	Method calls, local variables
Heap	Objects
Method Area	Class information
Native Method Stack	C/C++ code execution

One-Line Answer for Interview

PC Register:

> Stores the address of the currently executing instruction for each thread.

Native Method Stack:

> Stores and manages native methods written in C/C++ that are called from Java through JVM.

That's usually enough for a 3-year Java interview, bro. ✅

5. Execution Engine

Executes Bytecode.

Components:

Interpreter

Reads line by line.

```
Bytecode
    ↓
Machine Code
```

Slow.

JIT Compiler

Frequently used code:

```
Bytecode
    ↓
Compiled Once
    ↓
Machine Code
```

Fast.

Interview Question:

Why Java is faster nowadays?

Why Java is faster nowadays?

Answer:

> JVM uses JIT Compiler to compile frequently executed code into native machine code.

---

6. Garbage Collector

Inside Execution Engine.

Purpose:

```
Remove Unused Objects
``
```

Example:

```java
Citizen c = new Citizen();

c = null;
```

Object becomes eligible for GC.

Interview Answer:

> Garbage Collector automatically removes unused objects from heap memory.

- **Native Methods**: Enables interaction with non-Java code via JNI.

7. JNI

(Java Native Interface)

Allows Java to call native languages like:

```
1  C
2  C++
```

Difference between Stack and Heap?

Stack

- Local variables
- Method calls
- Faster

Heap

- Objects
- Shared memory
- Managed by GC

30-Second Interview Answer

> "Java code is compiled into bytecode by the javac compiler. The Class Loader loads the bytecode into JVM memory. Runtime Data Areas such as Heap, Stack, and Method Area store objects, local variables, and class metadata. The Execution Engine executes the bytecode using the Interpreter and JIT Compiler, while the Garbage Collector removes unused objects from heap memory."

🔥 For a **3-year Java developer**, focus on:

- Class Loader
- Heap
- Stack
- Method Area
- JIT Compiler
- Garbage Collector

These are asked in almost every Java interview. ✅

SOLID -->

S: Single Responsibility Principle (SRP)
"Instead of writing everything in one class, write responsibilities in different classes"

❌ Bad

```java
class CitizenService {

    void checkEligibility() {
    }

    void sendEmail() {
    }

    void generateReport() {
    }
}
```

This class has multiple responsibilities.

✅ Good

```java
class EligibilityService {

    void checkEligibility() {
    }
}

class EmailService {

    void sendEmail() {
    }
}

class ReportService {
```

Bro, **O = Open/Closed Principle (OCP)**

Simple Meaning

> **Classes should be Open for Extension, but Closed for Modification.**

Means:

✅ Add new functionality by creating new classes.

❌ Don't keep modifying existing working code.

Bad Example

```java
class PaymentService {

    void pay(String type) {
        if(type.equals("UPI")) {
            System.out.println("UPI Payment");
        }
        else if(type.equals("CARD")) {
            System.out.println("Card Payment");
        }
    }
}

```

Now business says:

> Add NetBanking.

Again you modify the same class.

Then Wallet.

Again modify.

This violates OCP.

Good Example

```java
interface Payment {
    void pay();
}

class UpiPayment implements Payment {
    public void pay() {
        System.out.println("UPI Payment");
    }
}

class CardPayment implements Payment {
    public void pay() {
        System.out.println("Card Payment");
    }
}
```

Later if NetBanking comes:

```java
class NetBankingPayment implements Payment {
    public void pay() {
        System.out.println("Net Banking Payment");
    }
}
```

✅ Existing code unchanged

✅ Just extend by adding a new class

S (SRP)

> Don't put everything in one class.

O (OCP)

> Don't change existing code; extend it with new classes.

Bro, **L = Liskov Substitution Principle (LSP)**

Simple Meaning

> **Child class should be able to replace Parent class without breaking the program.**

Real-Time Project Example

Java

```java
interface SchemeProcessor {
    void process();
}
```

Java

```java
class PMJAYScheme implements SchemeProcessor {
    public void process() {
        System.out.println("PMJAY Processing");
    }
}
```

Java

```java
class AarogyasriScheme implements SchemeProcessor {
    public void process() {
        System.out.println("Aarogyasri Processing");
    }
}
```

Anywhere:

Java

```java
SchemeProcessor processor = new PMJAYScheme();
```

or

Java

Easy Memory Trick

- **S** → One class = One responsibility
- **O** → Extend, don't modify
- **L** → Child should properly behave like Parent

Interview One-Liner

> Liskov Substitution Principle states that objects of a child class should be replaceable with objects of the parent class without affecting the correctness of the program.

Super simple remember:

👉 If `Parent obj = new Child();` works correctly everywhere, then LSP is followed, bro. ✅

Bro, **I = Interface Segregation Principle (ISP)**

Simple Meaning

> **Don't force a class to implement methods that it doesn't need.**

Real-Time Project Example

Instead of:

Java

```java
interface SchemeOperations {
    void createScheme();
    void updateScheme();
    void deleteScheme();
    void approveScheme();
}
```

Some classes may only need create/update.

So split:

Java

```java
interface CreateScheme {
    void createScheme();
}

interface UpdateScheme {
    void updateScheme();
}

interface ApproveScheme {
    void approveScheme();
}
```

A class implements only the required interfaces.

Easy Memory Trick

- **S** → One class = One responsibility
- **O** → Extend, don't modify
- **L** → Child should behave like Parent
- **I** → Don't force classes to implement unnecessary methods

Bro, **D = Dependency Inversion Principle (DIP)**

Simple Meaning

> **Depend on Interfaces, not Concrete Classes.**

Means:

✅ Use interface references.

❌ Directly create dependent objects inside the class.

Bad Example ❌

Java

```java
class MySqlDatabase {
    void save() {
        System.out.println("Saved");
    }
}

class EmployeeService {
    MySqlDatabase db = new MySqlDatabase();
}
```

Problem:

Tomorrow MySQL changes to Oracle.

Again modify EmployeeService

Good Example ✅

Java

```java
interface Database {
    void save();
}
```

Java

```java
class MySqlDatabase implements Database {
    public void save() {
        System.out.println("Saved in MySQL");
    }
}
```

Java

```java
class OracleDatabase implements Database {
    public void save() {
        System.out.println("Saved in Oracle");
    }
}
```

Java

```java
class EmployeeService {
    Database db;

    EmployeeService(Database db) {
        this.db = db;
    }
}
```

Easy Memory Trick for SOLID

S 👉 One class = One responsibility

O 👉 Extend code, don't modify existing code

L 👉 Child should behave like Parent

I 👉 Don't force unnecessary methods

D 👉 Depend on Interface, not Implementation

Interview One-Liner

> Dependency Inversion Principle states that high-level modules should not depend on low-level modules; both should depend on abstractions (interfaces).

Super Simple Remember

Solid:
S---Single Responsibility :Instead of writing everything in one class, write responsibilities in different classes"
O---> Open or close--->adding new functionilities on new class not an existing file
In the Integrity Eligibility System, when a new scheme is introduced, we create a new implementation class for that scheme instead of changing the existing eligibility logic. This follows the Open Closed Principle."

L - Liskov Substitution Principle (LSP)
A child class should be able to replace its parent class without breaking the application behavior.
ISP (Interface Segregation Principle):

Simple Definition

> **Don't force a class to implement unnecessary methods.**

Bad Example ❌

```java
interface SchemeService {

    void checkEligibility();

    void sendNotification();

    void generateReport();
}
```

Now:

```java
class PensionScheme implements SchemeService
```

must implement all 3 methods even if it only needs:

```java
checkEligibility();
```

I need only one method that check eligibility but we keep all methods in one interface every time I imnplement all methods so
Instead of one large interface, we should create multiple small and specific interfaces.
DIP (Dependency Inversion Principle).

SRP -> One Responsibility

OCP -> Add New Class, Don't Modify Old Class

LSP -> Child Can Replace Parent

ISP -> Don't Force Unnecessary Methods

DIP -> Depend On Interface, Not Implementation